

Mini-Project 2

Intelligent AI Agent

By: Ethan Ashton, Tris Replogle, Caitlyn Sherman, Nicholas Dias, Andre Saldi

Introduction

This project is the second project for the class. It requires students to create an agent using a chatbot. To do so the students will watch six videos to learn langchain to then use in an environment of their choice to make an agent. In this case, the team decided to try and make a study planner to use to plan for a future exam or quiz. Given the test files, the AI should be able to answer questions based on the information in the files.

Environment Description:

The AI in this project was run within a google Collab. Each member has access to it however it has very long runtimes; it tends to take around five to eight minutes to run all code blocks. We could separate each code block to run different parts of the code though which is why it was chosen. For the AI we used the OpenAI environment. This allows us to access a premade chatbot to call at any time. The environment for our agent can be described using the PEAS framework; this consists of performance measures, environment, actuators, and sensors. The performance of the agent is based on the accuracy and usefulness of the content it generates, the alignment with assignment deadlines and rubric requirements, and the clarity of the output. The environment includes documents provided by the user, the local file system where input and output are stored, and external knowledge accessed through the LLM and embedding models. Our agent is partially observable because it relies on available information and must infer any missing parts. The actuators of the agent include the ability to read files, write, generate excel spreadsheets, and produce textual responses. It can also retrieve information to make schedules. The sensors of the agent include PDF loaders that can extract text from provided documents, vector databases that allow semantic search, and user input prompts.

Architecture:

The AI Is built using the OpenAI architecture which takes care of most backend stuff to handle encryption and decryption. For the Agent we made the architecture would flow along the lines of a user-> interface-> local database-> OpenAI architecture-> local database->interface. The user would enter their input, the local database would store it, the input would be sent to the chatbot model ran by OpenAI, and then the database would receive the response and finally output it to the user.

Tools:

In this project, we have three tools implemented into the program. A directory management tool, a web searching tool, a PDF analysis tool, and an Excel chart tool. The directory management function, which uses the langchain file_management toolkit, allows the AI to read files, write files, and navigate through directories. The web searching tool works by importing the DuckDuckGo search function, which allows it to make searches and read through the results. The PDF tool works by teaching an LLM using all of the text contained within all the PDF files within the directory, then when the function is called it allows the Agent to ask it questions to the

trained LLM. The Excel chart works using the `xlsxwriter` library and prompts the agent to input various information which is then compiled into an excel file and creating a graph.

Memory:

The memory works by saving instances of the previous interactions as a dictionary which the AI can review. The short-term memory is fed in context to requests. The long-term memory is stored as threads which are fed into the model at the start of an interaction.

Rationality:

This agent is meant to be used to plan out a set of steps that will help the user to plan their studying. To do this, the agent waits for the user to tell it what they need; the agent then chooses what function it believes best fits the given task. For example, if the agent received a question like when assignment 2 is due, it will call `gather` to gather the information and respond to the user. The agent then reads through each given file and finds any information relevant to the question. When they find the information they need, the agent will then formulate a response with the information found. For example, if the user starts out by asking for the date of assignment 1, the AI will search the documents to find the due date for assignment 1 and return the information to the user. If the user asks the AI to help them plan out studying for the midterm, the AI will search the documents, find the date of the exam, and any notes there are and create a plan to help study for the exam. This makes the agent rational as it is deciding things based on information it is given.

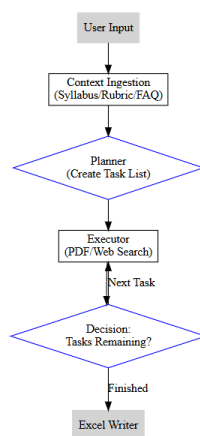


Figure 1: the steps taken by AI

Limitations:

Our main limitations for this project were the time we all had available. Given that we are taking many different classes with different projects and exams, it was difficult to find time to work on the project. There was not really time to learn the tools needed for the project very well as we were not given much time following the first project. The AI is limited in how it can respond due

to the information given to it. If it cannot find any information in a file about a topic, it cannot really create a response to the user about it. Outside of that, it can only handle two different file types, pdf and excel. While it can search the web, the information it might need might not be on there. This means the user will have to accept that the AI might not be able to tell them when a pop quiz may happen.

Conclusion

The agent for this project was run in google Collab and used OpenAI for the AI response. The tools for the agent were a directory management tool, a web searching tool, a PDF analysis tool, and an Excel chart. The memory is handled by saving past interactions as a dictionary for the agent to review. The AI is limited by only knowing what assignments there are based on the files it is given. This project helped us learn about making an agent and how much time and effort it can take to implement the tools and procedures.